

Section C: Development

TECHNIQUES USED TO IMPLEMENT THE SOLUTION

Class Diagram:.....	2
GUI: Authentication, Event Handling, Window Listener:	3
Inheritance	3
User Interface	4
Method Override (Polymorphism) and GUI dialogue	5
Menu Driven EMS:	7
Encapsulation, Inputs and Outputs.....	7
Conditional statements.....	9
Control Structures.....	10
Multi-threads	10
Exception Handling	12
Try-catch statement.....	12
MySQL, JDBC, SQL	14
Integrating an RDBMS with Java	14
Two-dimensional Arrays:	18
Using JTable	19
External Packages	20
Abbreviations	22

Class Diagram:

UML Class Diagram for EMS

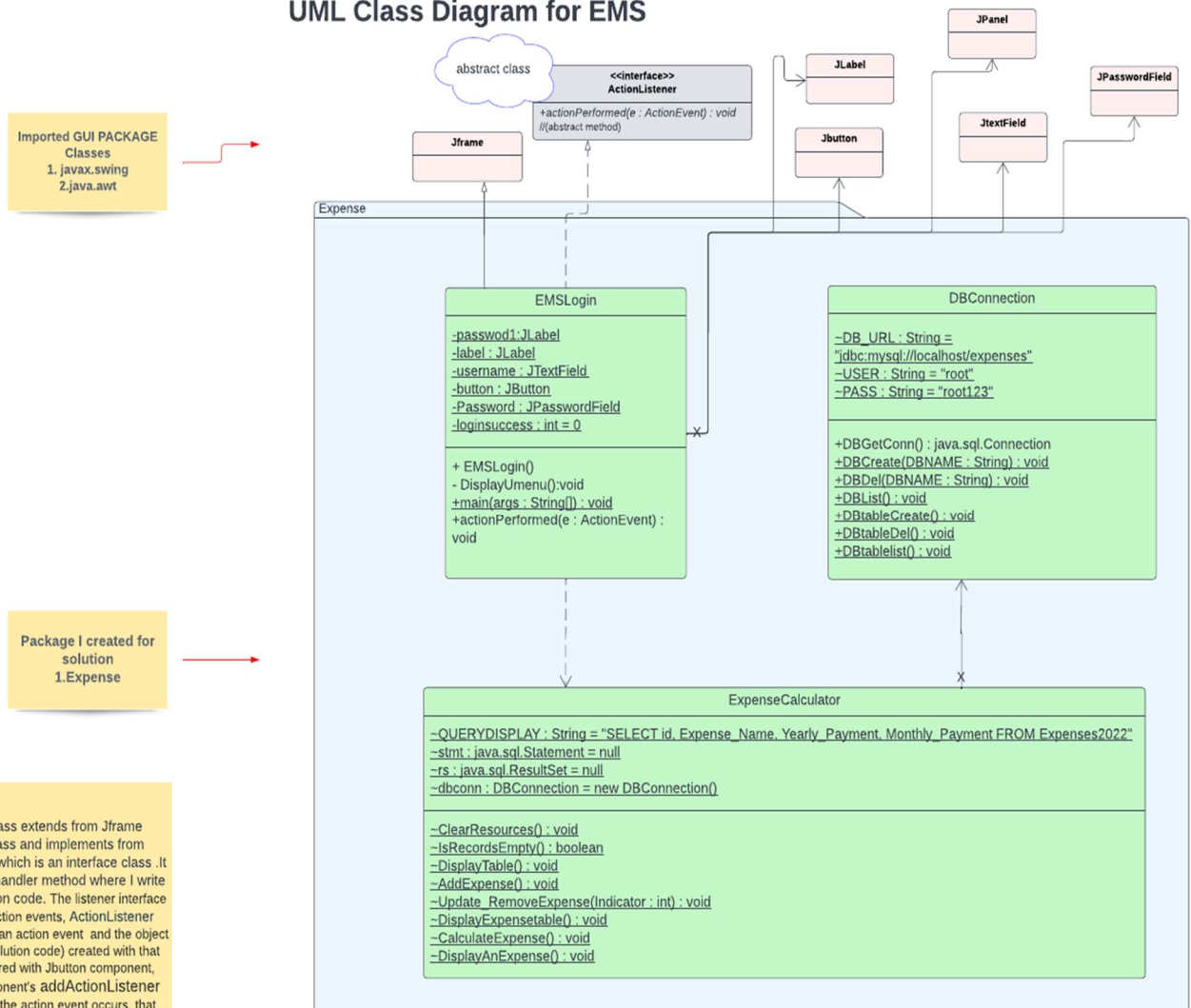
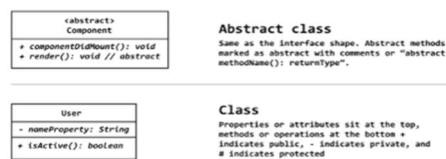


Fig 1



UML HELP

GUI: Authentication, Event Handling, Window Listener:

Success Criteria 1, 2, 3, 4

In my Expense Management system (EMS) solution a login GUI interface appears at start-up.

My solution checks credentials entered allowing only authorized logins to access the EMS

Menu. I created this to provide a level of security.

In the **Class EMSLogin**:

- I imported the Java GUI packages (Refer class diagram):

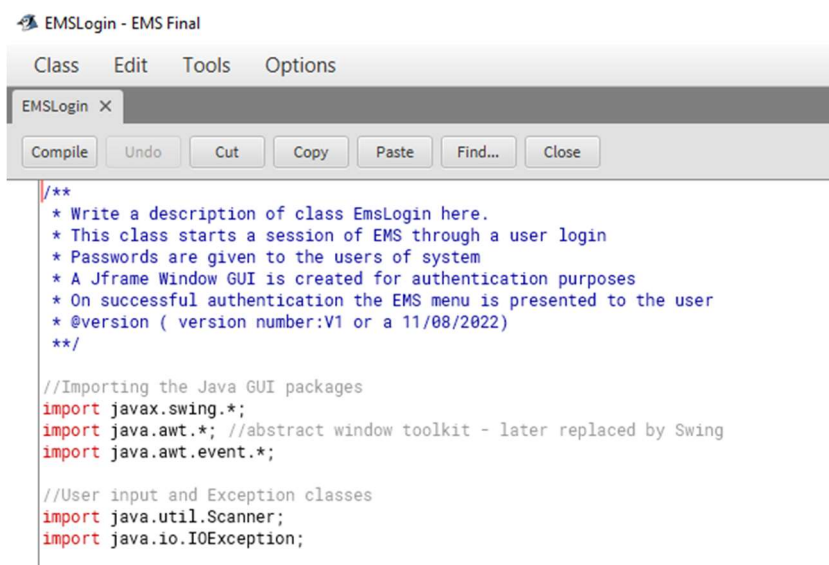


Figure 1

Inheritance

- I made **EMSLogin** inherit JFrame (*Java swing class*). I encapsulated the variables to enter login credentials to protect these variables from being manipulated outside this class. I made **the class** implement an *ActionListener class* (from java.awt.event package), to listen to the **login button click events** on the login GUI.

```
public class EMSLogin extends JFrame implements ActionListener {
    // Class Variables
    private static JLabel password1, label;
    private static JTextField username;
    private static JButton button;
    private static JPasswordField Password;
}
```

Figure 2

User Interface

- I made the constructor of **EMSLogin** to create a *JFrame* Window. This is the main login GUI window holding all the components. I made the button object register to events (highlighted code below).

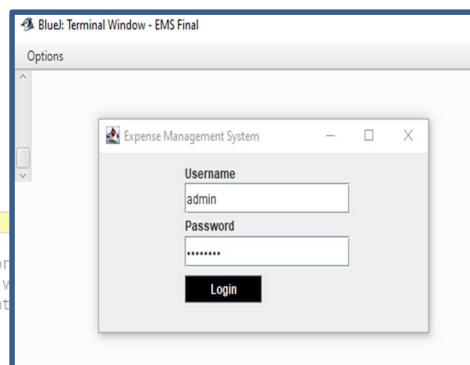


Figure 3

```
public EMSLogin() {
    /*Adding JPanel to the window
    JPanel class covers the entire space of
    Instantiates a JPanel class using the v
    meaning: the layout should take the ent
    */

    // creating a JPanel class
    JPanel panel = new JPanel(null);

    //Adding JFrame :Frame class that contains all the components that we see on the screen
    //Set JFrame parameters
    //Setting the JFrame title name that will appear at the top of the frame
    setTitle("Expense Management System");

    //Positioning the frame on the window - 500 pixels from the left, 300 pixels from the top of the screen
    setLocation(new Point(100, 100));

    //Declares the frame on the panel
    add(panel);

    //Sets the size of the entire frame - 400 pixels wide (width), 200 pixels long (height)
    setSize(new Dimension(400, 200));

    //This will end and close the process whenever the user exits the window
    setDefaultCloseOperation(HIDE_ON_CLOSE);

    //Username Label constructor
    label = new JLabel("Username");
    label.setBounds(100, 8, 70, 20);
    panel.add(label);

    //Username TextField constructor
    username = new JTextField();
    username.setBounds(100, 27, 193, 28);
    panel.add(username);

    //Password Label constructor
    password1 = new JLabel("Password");
    password1.setBounds(100, 55, 70, 20);
    panel.add(password1);

    //Password TextField
    Password = new JPasswordField();
    Password.setBounds(100, 75, 193, 28);
    panel.add(Password);

    //Button constructor
    button = new JButton("Login");
    button.setBounds(100, 110, 90, 25);
    button.setForeground(Color.WHITE);
    button.setBackground(Color.BLACK);
    button.addActionListener(this);

    panel.add(button);
    setAlwaysOnTop(true);
    setVisible(true);
    System.out.println("I am here in the EMS login\n\n");
}
```

Figure 4

Method Override (Polymorphism) and GUI dialogue

- I wrote the authentication code in the *ActionPerformed* method of the *ActionListener listener class* which will be called whenever the login button is clicked.
- I used the *JOptionPane* dialogue component of Java GUI, to display either a warning, success, or error message to the user. I used 'get' and 'set' methods to access the private variables of the Class EMSlogin within itself.

```
/*
 * Authentication: This method will be invoked when we click on the Login button. It helps authenticate the data entered in the username and password fields.
 * Implementing an action listener class with conditional statement
 * JOptionPane is another component of Java GUI, it comes in handy when displaying either a warning, success, or error message on the screen, as a dialog box.
 */
@Override
public void actionPerformed(ActionEvent e) {
    String Username = username.getText();
    String Password1 = Password1.getText();

    if (Username.equals("admin") && Password1.equals("admin123"))
    {
        JOptionPane.showMessageDialog(null, "Login Successful, You are the " + Username + "", "Login", JOptionPane.PLAIN_MESSAGE);
        setVisible(false);
        loginsuccess = 1;
    }

    else if (Username.equals("user") && Password1.equals("user123"))
    {
        JOptionPane.showMessageDialog(null, "Login Successful!", "Login", JOptionPane.PLAIN_MESSAGE);
        setVisible(false);
        loginsuccess = 1;
    }

    else{
        int result = JOptionPane.showConfirmDialog(null, "Username or Password mismatch, Try Again ", "Login alert!", JOptionPane.YES_NO_OPTION, JOptionPane.QUESTION_MESSAGE);
        if(result == JOptionPane.YES_OPTION){
        }else if (result == JOptionPane.NO_OPTION){
            setVisible(false);
            dispose();
            loginsuccess = 2;
            System.exit(0);
        }
        else if (result == JOptionPane.CLOSED_OPTION) {
            System.out.println("Window closed without selecting!");
            loginsuccess = 2;
            System.exit(0);
        }
    }
}
}
```

Figure 3a

- I added a window listener to login frame where I update the loginsuccess variable to address the multithreading problem mentioned later in the control section.

```
//This will end and close the process whenever the user exits the window
setDefaultCloseOperation(HIDE_ON_CLOSE);

addWindowListener(new WindowAdapter() {
    public void windowClosing(WindowEvent we) {
        if(loginsuccess == 0 || loginsuccess == 2){
            //System.out.println("Debug: I am here inside if close\n" +loginsuccess);
            System.exit(0);
        }
        // System.out.println("Debug: I am here outside if close\n"+loginsuccess);
    }
});
```

Figure 4b

The java™ tutorials. Available at: <https://docs.oracle.com/javase/tutorial/> (Accessed: August 15, 2022).

How to Write an Action Listener (The Java™ Tutorials > Creating a GUI With Swing > Writing Event Listeners). Available at: <https://docs.oracle.com/javase/tutorial/uiswing/events/actionlistener.html> (Accessed: August 16, 2022).

Lesson: Using Swing Components (The Java™ Tutorials > Creating a GUI With Swing). Available at: <https://docs.oracle.com/javase/tutorial/uiswing/components/index.html> (Accessed: August 16, 2022).

JFrame (Java Platform SE 7). Available at: <https://docs.oracle.com/javase/7/docs/api/javax/swing/JFrame.html> (Accessed: August 16, 2022).

Java JSCROLLPANE – Javatpoint, www.javatpoint.com. Available at: <https://www.javatpoint.com/java-jscrollpane> (Accessed: August 16, 2022).

Introduction to computer science using Java. Available at: <https://chortle.ccsu.edu/java5/index.html#57> (Accessed: August 16, 2022).

Hameed, M. *et al.* (2022) *Login form in Java Swing with source code tutorial: Login page, Tutorials Field*. Available at: <https://www.tutorialsfeld.com/login-form-in-java-swing-with-source-code/> (Accessed: August 16, 2022).

Login form Java – Javatpoint, www.javatpoint.com. Available at: <https://www.javatpoint.com/login-form-java> (Accessed: August 16, 2022).

Menu Driven EMS:

Success Criteria 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 20

On successful authentication, my solution will present the EMS menu to the authorized user.

Encapsulation, Inputs and Outputs

- I created a **private method** (restricting its access for other classes or subclasses) for the EMSLogin class to display the menu options and the output based on the option selected by the user. I used the System Class with its field 'out' to print options.

```
private void DisplayUmenu() throws Exception
{
    //Declaring variables
    int choice;

    //Creating an infinite while loop so control is returned back to the EMS main menu
    //after the appropriate function is called based on a choice made by the user
    while(true) {
        System.out.println("\n-----");
        System.out.println("-----          MAIN MENU          -----");
        System.out.println("-----");

        //Creating menu
        System.out.println("\n- Press 1 to Add Expense                -");
        System.out.println("- Press 2 to Remove Expense                -");
        System.out.println("- Press 3 to Update Expense                -");
        System.out.println("- Press 4 to Display Expense Table        -");
        System.out.println("- Press 5 to Display Yearly/Monthly Expense -");
        System.out.println("- Press 6 to Display an Expense");
        System.out.println("- Press 7 to Quit\n\n ");
        System.out.println("-----");
        System.out.println("-----");
        System.out.println("-----");

        //Asking the user to make choice
        System.out.println("Make your Choice");
        Scanner sc = new Scanner(System.in);

        //Checking for an invalid input choice
        if(sc.hasNextInt()) {
            choice = sc.nextInt();

            //This variable is sent as a argument to Update_RemoveRecord
            //if its value is 0, it will remove a record
            //if its value is 1, it will update a record.
            int edit = 0;

            //Creating a switch case branch based on the choice made by the user
            switch (choice) {
                //First case adds the record to the user's expense table
                case 1:
                    //System.out.println("CALLING AddExpense() ");
                    ExpenseCalculator.AddExpense();
                    break;

                //Second case removes a record from the expense table
                case 2:
                    //System.out.println("Calling RemoveExpense() ");
                    ExpenseCalculator.Update_RemoveExpense(edit);
                    break;
            }
        }
    }
}
```



```

//Third case updates a record in the expense table
case 3:
    // System.out.println("Calling UpdateAnExpense()");
    edit=1;
    ExpenseCalculator.Update_RemoveExpense(edit);
    break;

//Fourth case displays the expense table
case 4:
    // System.out.println("Calling DisplayTable()");
    ExpenseCalculator.DisplayTable();
    break;

//Fifth case calculates the total amount of expenses and displays it
case 5:
    //System.out.println("Calling CalculateExpense()");
    ExpenseCalculator.CalculateExpense();
    break;

//Sixth case outputs the following statement
case 6:
    //System.out.println("Calling CompareExpenses() will be implemented in release 2 as an extension of this project");
    ExpenseCalculator.DisplayAnExpense();
    break;

//Seventh case quits the program
case 7:
    System.out.println("Exiting the EMS.\n\n");
    System.exit(0);

//A default case which displays the message: invalid choice made by the user
default:
    System.out.println("Invalid Choice!!! Please enter a Valid Menu Choice.\n\n");
}
}
else {
    System.out.println("Invalid Choice!!! Please enter a Valid integer for Menu Choice. \n \n");
}
}
}

```

Figure 5

- I used the **scanner class object** to take input choices from the user as it's a simple way to get an input from the keyboard. I used the **scanner class's** data validation method **hasNextInt()** to check for integer inputs.

```

Scanner user_input = new Scanner(System.in);
double YPayment = 0;
double MPayment = 0;
String Ex_name;

int Confirm;

System.out.println("Press 1 to Enter Monthly Payment");
System.out.println("OR");
System.out.println("Press 2 to Enter Yearly Payment");

if(user_input.hasNextInt()) {
    Confirm = user_input.nextInt();
    while((Confirm !=1 ) && (Confirm !=2))
    {

```

Figure 6

Conditional statements

In my solution I wrote:

- If-else, default switch-case for data validation checks.
- *Switch-cases* to call appropriate methods to print the output based on the user's menu choice. I used it as it's easier to debug, maintain and for better clarity. It is the best-optimized implementation for faster code execution than the "if-else if" statement.

```
//Checking for an invalid input choice
if(sc.hasNextInt()) {
    choice = sc.nextInt();

    //This variable is sent as a argument to Update_RemoveRecord
    //if its value is 0, it will remove a record
    //if its value is 1, it will update a record.
    int edit = 0;

    //Creating a switch case branch based on the choice made by the user
    switch (choice) {
        //First case adds the record to the user's expense table
        case 1:
            System.out.println("CALLING AddExpense() ");
            ExpenseCalculator.AddExpense();
            break;

            //Second case removes a record from the expense table
        case 2:
            System.out.println("Calling RemoveExpense() ");
            ExpenseCalculator.Update_RemoveExpense(edit);
            break;

            //Third case updates a record in the expense table
        case 3:
            System.out.println("Calling UpdateAnExpense()");
            edit=1;
            ExpenseCalculator.Update_RemoveExpense(edit);
            break;

            //Fourth case displays the expense table
        case 4:
            System.out.println("Calling DisplayTable()");
            ExpenseCalculator.DisplayTable();
            break;

            //Fifth case calculates the total amount of expenses and displays it
        case 5:
            System.out.println("Calling CalculateExpense()");
            ExpenseCalculator.CalculateExpense();
            break;

            //Sixth case outputs the following statement
        case 6:
            System.out.println("Calling CompareExpenses() will be implemented in release 2 as an extension");
            ExpenseCalculator.CompareExpenses();
            break;

            //Seventh case quits the program
        case 7:
            System.exit(0);

            //A default case which displays the message: invalid choice made by the user
        default:
            System.out.println("Invalid choice!!! Please make a valid choice. \n\n");
    }
}

else {
    System.out.println("Invalid choice!!! Make a valid integer Choice. \n\n");
}
```

Figure 7

Control Structures

- I created an infinite *while loop*, so control is returned back to the EMS main menu after the appropriate output is displayed unless the user selects '7' to exit.

```
while(true) {
    System.out.println("-----");
    System.out.println("-----      MAIN MENU      -----");
    System.out.println("-----");

    //Creating menu
    System.out.println("\n-   Press 1 to Add Expense           -");
    System.out.println("-   Press 2 to Remove Expense       -");
    System.out.println("-   Press 3 to Update Expense       -");
    System.out.println("-   Press 4 to Display Expense Table -");
    System.out.println("-   Press 5 to Display Yearly/Monthly Expense -");
    System.out.println("-   Press 6 to Compare Expense");
    System.out.println("-   Press 7 to Quit\n\n ");
    System.out.println("-----");
    System.out.println("-----");
    System.out.println("-----");
}
```

Figure 8

Multi-threads

- I used *Do-while* along with a *variable (loginsuccess)* to control the **multithread situation** for control flow. When the main method creates an instance of EMSLogin, a new thread is started independent of the main method which displays login GUI. To avoid displaying the menu simultaneously at this point, I needed to **control the events** through the *loginsuccess* variable.

```
public class EMSLogin extends JFrame implements ActionListener {
    // Class Variables
    private static JLabel password1, label;
    private static JTextField username;
    private static JButton button;
    private static JPasswordField Password;
    //This variable is needed to handle the multithread situation for control flow.
    private static int loginsuccess = 0;
```

Figure 9

```

public static void main(String[] args) throws Exception{
    try
    {
        //create instance of the CreateLoginForm
        EMSLogin myuser = new EMSLogin();
        System.out.println("login " +loginsuccess);
        System.out.println("Welcome to EMS, Waiting for Authentication");
        //wait;
        do{
            // System.out.println("login inside do while" +loginsuccess);
            System.out.println(" ");
        }
        while(loginsuccess==0);

        System.out.println("login outside do while" +loginsuccess);
        if(loginsuccess == 1){
            // System.out.println(System.lineSeparator().repeat(100));
            System.out.println(" ");
            System.out.print('\u000C');
            myuser.DisplayUmenu();
        }
    }
}

```

Figure 10

Exception Handling

Try-catch statement

Success Criteria 6, 7, 8

- My solution provides appropriate error messages throughout the application on runtime errors using **try-catch methods** of the Exception class. Thus, leaving helpful notices for the user as to what went wrong, prompting them to re-input in the correct format.

Eg:

```
//User input and Exception classes
import java.util.Scanner;
import java.io.IOException;
```

Figure 11

```
public static void main(String[] args) throws Exception{
    try
    {
        //create instance of the CreateLoginForm
        EMSLogin myuser = new EMSLogin();
        System.out.println("login " +loginsuccess);
        System.out.println("Welcome to EMS, Waiting for Authentication");
        //wait;
        do{
            // System.out.println("login inside do while" +loginsuccess);
            System.out.println(" ");
        }
        while(loginsuccess==0);

        System.out.println("login outside do while" +loginsuccess);
        if(loginsuccess == 1){
            // System.out.println(System.lineSeparator().repeat(100));
            System.out.println(" ");
            System.out.print('\u000C');
            myuser.DisplayUmenu();
        }
    }
    catch(Exception e)
    {
        //handle exception
        JOptionPane.showMessageDialog(null, e.getMessage());
    }

    System.out.println("I am here in main\n\n");
    System.out.println("login " +loginsuccess);
}
```

Figure 13

```
}
try{
    for (int d = 0; d < Array_Expenses.length; d++) { //this equals to the row in our
        for (int j = 0; j < Array_Expenses[d].length; j++) { //this equals to the col
            System.out.print(Array_Expenses[d][j] + " ");
        }
        System.out.println();
    }
}
catch(Exception e){
    System.out.print("Error : " +e);
}
```

Figure 14

```
try {  
    String pattern = "yyyy/mm/dd";  
    SimpleDateFormat formatter = new SimpleDateFormat(pattern);  
    //Now use the parse() method to convert the date in text form  
    Date Sst_dateObj = formatter.parse(Sst_date);  
    System.out.println(Sst_date);  
    System.out.println(Sst_dateObj);  
}  
catch (ParseException e) {  
    System.out.print("wrong date format");  
    return;  
}
```

Figure 15

MySQL, JDBC, SQL

Integrating an RDBMS with Java

Success Criteria 8, 9, 16, 19

My solution uses **MySQL RDBMS** to store the expense table as the client wanted to store multiple tables in the future and relate them (**Appendices A, B**). It's easy to use, compatible with Java, and a reliable open source. It provides a way to create, maintain and control access to database files/tables. The database design optimizes the way in which stored data can be structured, manipulated, and retrieved in a secure manner using SQLs which is versatile and flexible.

- In my solution, the EMS connects to MySQL using Java™ database connectivity (JDBC) APIs and uses **SQL queries** to manage MySQL DB table.

The steps I followed for MYSQL Connection and access:

- I imported the **JDBC classes packages** for database programming:

```
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.SQLException;  
import java.sql.Statement;  
import java.sql.ResultSet;
```

Figure 16

- **Database URL Formulation** pointing to the database in MySQL I want to connect.

```
static final String DB_URL = "jdbc:mysql://localhost/Expenses"; //table location  
static final String DB_URL1 = "jdbc:mysql://localhost/";  
static final String USER = "root";  
static final String PASS = "root123";
```

Figure 17

- I used **DriverManager.getConnection()** method to create a JDBC **Connection** object, which represents a physical connection with the **MySQL** server.

```
// Open a connection
Connection conn = null;
System.out.println("Inside dbGETCONN block");
try
{
    // Registering drivers
    conn = DriverManager.getConnection(DB_URL, USER, PASS);
    System.out.println("Inside try block");
    System.out.println("Connected to database");
}
catch (SQLException e) {
    e.printStackTrace();
    System.out.println("Inside catch block");
}
System.out.println("Outside try/catch block");
return conn;
```

Figure 18

- I used an object of type **JDBC Statement** for building and submitting SQL queries to the database.

```
try(Connection conn = DriverManager.getConnection(DB_URL1, USER, PASS);
Statement stmt = conn.createStatement();)
{
    String sql = "CREATE DATABASE " + DBNAME + " ";
    stmt.executeUpdate(sql);
    System.out.println("Database created successfully...");
}
catch (SQLException e) {
    e.printStackTrace();
    System.out.println("Inside catch blockDB CREATE");
}
System.out.println("Outside try/catch db create block");
```

Figure 19

- I used an object of type **JDBC ResultSet** to hold the results from executing an SQL query. **ResultSet** is retrieved through a set of **get**, **next** methods and **while loop** to iterate through the records of the table.


```

QUERYDELUP = "Select * from expenses2022 " + "WHERE expense_Name='" + Ex_name + "' ";
stmt = dbconn.DBGetConn().createStatement();
//Executing the query
rs = stmt.executeQuery(QUERYDELUP);
while(rs.next()){
    //Display values
    System.out.println("    EXPENSE NAME: " + rs.getString("expense_Name"));
    System.out.println("    START DATE: " + rs.getDate("start_date"));
    System.out.println("    YEARLY PAYMENT: " + rs.getDouble("Yearly_Payment"));
    System.out.println("    MONTHLY PAYMENT: " + rs.getDouble("Monthly_Payment"));
    System.out.println("    EXPIRY DATE: " + rs.getDate("expiry_date"));
}

```

Figure 20

- The method 'IsRecordsEmpty' in my solution does as a validity check to see if the DB is empty before attempting to edit it. I used a **For loop** to iterate through the **ResultSet** object to print column names if records are empty in the DB.

```

//this functions returns true if there are no records in the DB
static boolean IsRecordsEmpty()throws Exception
{
    System.out.println("Static method IsRecordsEmpty");
    //We could have simply used count(*) to get number of records but to print column na
    String QUERYDISPLAY = "SELECT * FROM expenses2022";
    stmt = dbconn.DBGetConn().createStatement();
    //Executing the query
    rs = stmt.executeQuery(QUERYDISPLAY);

    //Retrieving the result
    if (!rs.next()) {
        System.out.print("No Expense Records in Datable !Please add the expenses!");
        // Get column names from table using ResultSetMetaData
        //https://docs.oracle.com/javase/8/docs/api/java/sql/ResultSetMetaData.html
        ResultSetMetaData rsmd = rs.getMetaData();
        int columnCount = rsmd.getColumnCount();
        System.out.println("Printing the columns in our Expense2022 table : ");
        // Get the column labels; column indices start from 1
        for (int i=1; i<=columnCount; i++) {
            String columnName = rsmd.getColumnLabel(i);
            System.out.println(columnName);
        }
        return true;
    }
    return false;
}

```

Figure 21

- The solution cleans up the resources after use with **close()**, aiding memory

management.

```
if (rs != null) {  
    try {  
        rs.close();  
    } catch (SQLException sqlEx) { }  
    rs = null;  
}  
  
if (stmt != null) {  
    try {  
        stmt.close();  
    } catch (SQLException sqlEx) { }  
    stmt = null;  
}
```

MySQL 8.0 Reference Manual, MySQL. Available at: <https://dev.mysql.com/doc/refman/8.0/en/> (Accessed: August 22, 2022).

Java & MySQL Tutorial, Tutorials Point. Available at: https://www.tutorialspoint.com/java_mysql/index.htm (Accessed: August 22, 2022).

JDBC resultset: How to use java RESULTSET to retrieve data (2023) Software Testing Help. Available at: <https://www.softwaretestinghelp.com/jdbc-resultset-tutorial/> (Accessed: August 22, 2022).

MySQL Connector/J 8.0 Developer Guide :: 7.2 using JDBC statement objects to execute SQL, MySQL. Available at: <https://dev.mysql.com/doc/connector-j/8.0/en/connector-j-usagenotes-statements.html> (Accessed: August 22, 2022).

MySQL Workbench 8.0.32, MySQL. Available at: <https://dev.mysql.com/downloads/workbench/> (Accessed: August 22, 2022).

Two-dimensional Arrays:

Success Criteria 8, 9, 13, 14, 16, 17

My solution reads the MySQL DB table and display its data in the form of a Java GUI table.

To implement this, I created a 2d Array 'Array_Expenses' of string type. A 2D Array was appropriate as it allowed me to populate its row value by querying the row count from the MySQL DB and column count 6 as it's fixed. I then initialized the value of 'Array_Expenses' from the row values of DB table got from the SQL queries.

```
String[][] Array_Expenses;
//Executing the query
int Array_row =0;

rs = stmt.executeQuery(QUERYROWS);
if (rs.next()) {
    /* this is important to get row numbers from mysql table to initialise our 2D arrays
    * we have 6 columns currentlyyyyyy ,refering the mysql create table
    */
    Array_row = rs.getInt(1);
    System.out.println("Row Count " + rs.getInt(1) + " " + Array_row );
    Array_Expenses = new String[Array_row][5];

    String QUERY = "SELECT * FROM expenses2022";
    stmt = dbconn.DBGetConn().createStatement();
    //Executing the query
    rs = stmt.executeQuery(QUERY);
    int i=0;
    while(rs.next()){
        //Display values

        //Converting Date object to String
        Date sdate = rs.getDate("Start_Date");
        String s_date = sdate.toString();
        // System.out.println("Date: "+sdate.toString());

        Date edate = rs.getDate("expiry_date");
        String e_date = edate.toString();
        //System.out.println("Date: "+edate.toString());
        //String.valueOf(rs.getInt(1))

        Array_Expenses[i][0] = rs.getString("expense_Name");
        System.out.print(Array_Expenses[i][0]+" ,");
        Array_Expenses[i][1] = s_date;
        System.out.print(Array_Expenses[i][1]+" ,");
        Array_Expenses[i][2] = String.valueOf(rs.getDouble("Yearly_Payment"));
        System.out.print(Array_Expenses[i][2]+" ,");
        Array_Expenses[i][3] = String.valueOf(rs.getDouble("Monthly_Payment"));
        System.out.print(Array_Expenses[i][3]+" ,");
        Array_Expenses[i][4] = e_date;
        System.out.print(Array_Expenses[i][4]);
        i++;
        System.out.println(); //change line on console as row comes to end in the matrix.
        // System.out.println("Hi"+i);
    }
}
```

Figure 23

How to set multidimensional array into jtable with java, Tutorials Point. Available at: <https://www.tutorialspoint.com/how-to-set-multidimensional-array-into-jtable-with-java> (Accessed: August 24, 2022).

Learn java: Two-dimensional arrays Cheatsheet, Codecademy. Available at: <https://www.codecademy.com/learn/learn-java/modules/java-two-dimensional-arrays/cheatsheet> (Accessed: August 24, 2022).

Using JTable

Success Criteria 16, 19, 20

To display the expense table data from MySQL DB my solution uses an object of Jtable class (Java GUI swing Component). I passed the 'Array_Expenses' and a single dimension array with column names as arguments to the constructor of Jtable and passed the Jtable object 'table' to populate JFrame window (**figure 24**).

I used Jtable as it creates a simple data view with easy readability and has the ability of editing tables of cells which I want to use in a future release (**Appendix I**).

```
JTable table = new JTable(Array_Expenses,new String[] {"Expense_Name","Start_Date","Yearly_Payment","Monthly_Payment","Expiry_Date"});
Font font = new Font("Verdana", Font.PLAIN, 12);
table.setFont(font);

JFrame frame = new JFrame();
frame.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
//frame.setIconImage(new ImageIcon(imgURL).getImage());
frame.setTitle("My Expenses");
frame.setSize(900, 600);
frame.add(new JScrollPane(table));
// Set's the window to be "always on top"
frame.setAlwaysOnTop( true );

//this will oveide the setSize to adjustable view table
table.setPreferredScrollableViewportSize(table.getPreferredSize());
JScrollPane scrollPane = new JScrollPane( table );
frame.add(scrollPane);
frame.pack();

//Displaying the window
frame.setVisible(true);
}
```

Figure 23

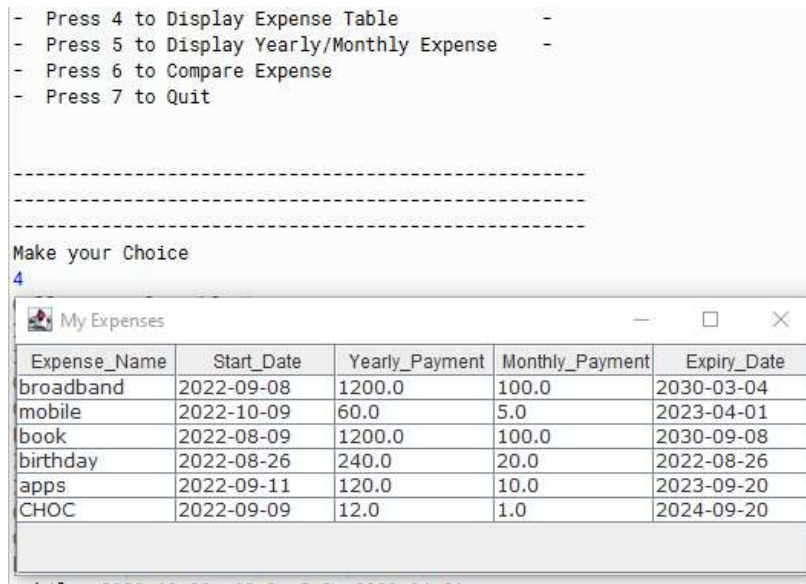


Figure 24

docs.oracle.com. (n.d.). *How to Use Tables (The Java™ Tutorials > Creating a GUI With JFC/Swing > Using Swing Components)*. [online] Available at: <https://docs.oracle.com/javase/tutorial/uiswing/components/table.html> [Accessed 24 Aug. 2022].

External Packages

The external packages in my solution helped me simplify and speed up the development process. These libraries contain pre-written code which accommodated the user interface, design inputs/outputs, error handling DB connections and management.

```

//For JDBC
import java.sql.SQLException;
import java.sql.ResultSet;
import java.sql.ResultSetMetaData;
import java.sql.Statement;

//Date formatting
import java.text.SimpleDateFormat;
import java.util.Date;
import java.text.ParseException;

// to get user input,
import java.util.Scanner;

//i/o exception
import java.io.IOException;

//for JAVA GUI and Java GUI Table
import java.awt.Font;
import javax.swing.JFrame;
import javax.swing.JLabel;

import javax.swing.JScrollPane;
import javax.swing.JTable;

// For MySQL connectivity
import java.sql.Connection;
import java.sql.DriverManager;

```

Figure 25

Abbreviations

Abbreviation	Definition
2D	Two Dimensional
API	Application programming interface
DB	Database
EMS	Expense Management System
GUI	Graphical User Interface
JDBC	Java™ database connectivity
MYSQL	Its name is a combination of "My", the name of co-founder Michael Widenius's daughter My, and "SQL", the abbreviation for Structured Query Language.
RDBMS	Relational Database Management System
SQL	Structured Query Language
URL	Uniform Resource Locator

Figure 26

1,058 words